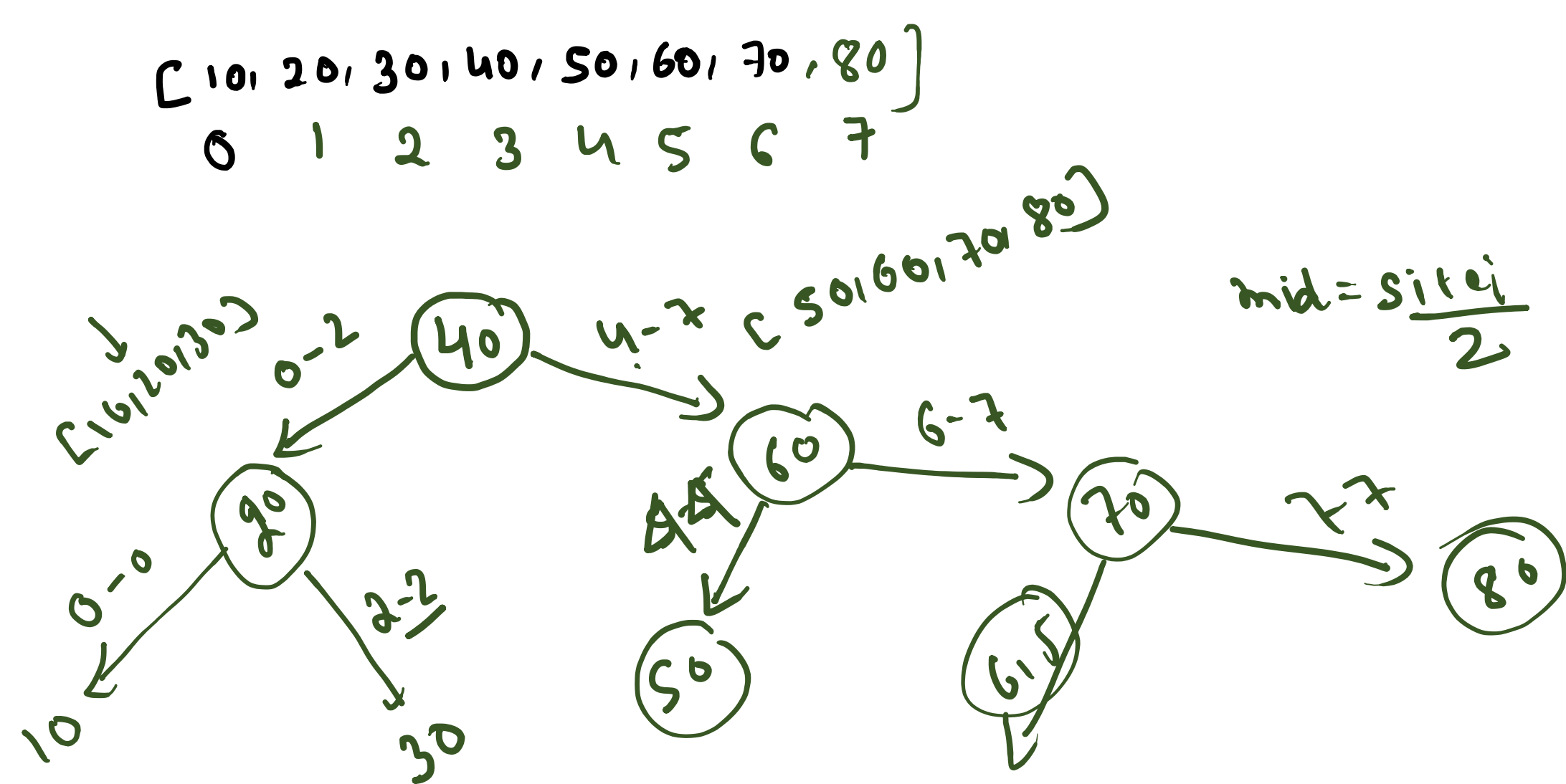
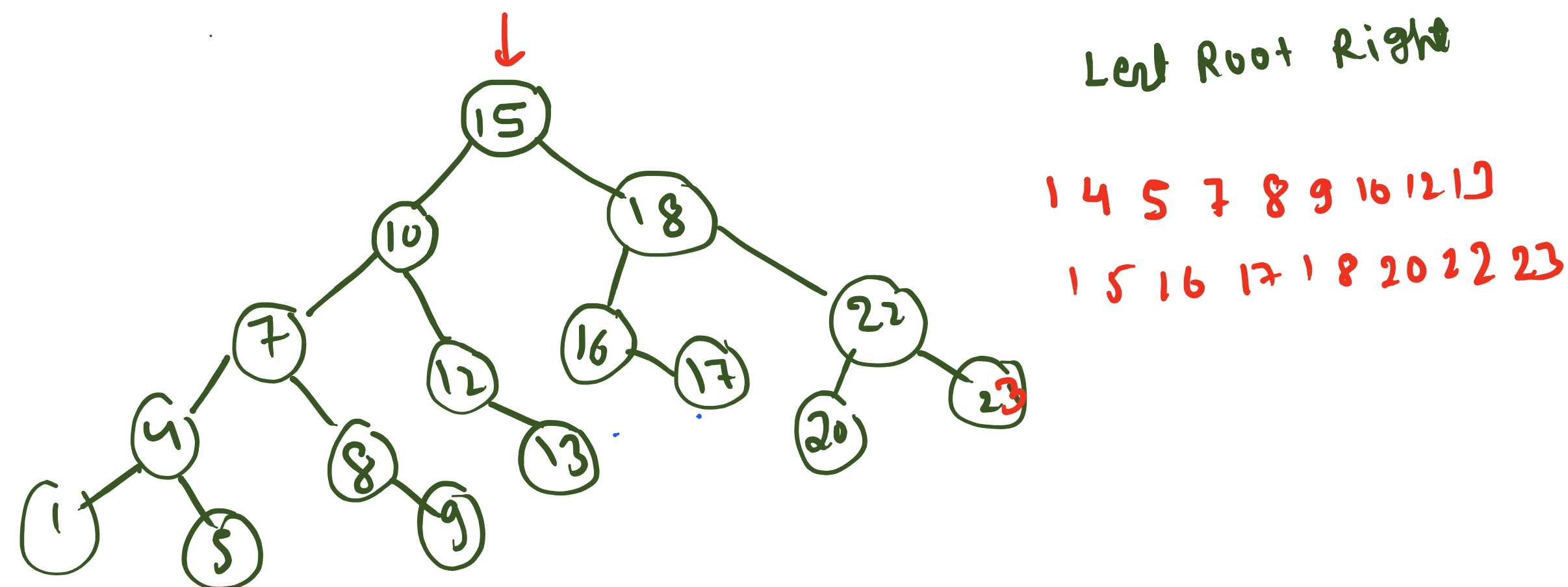
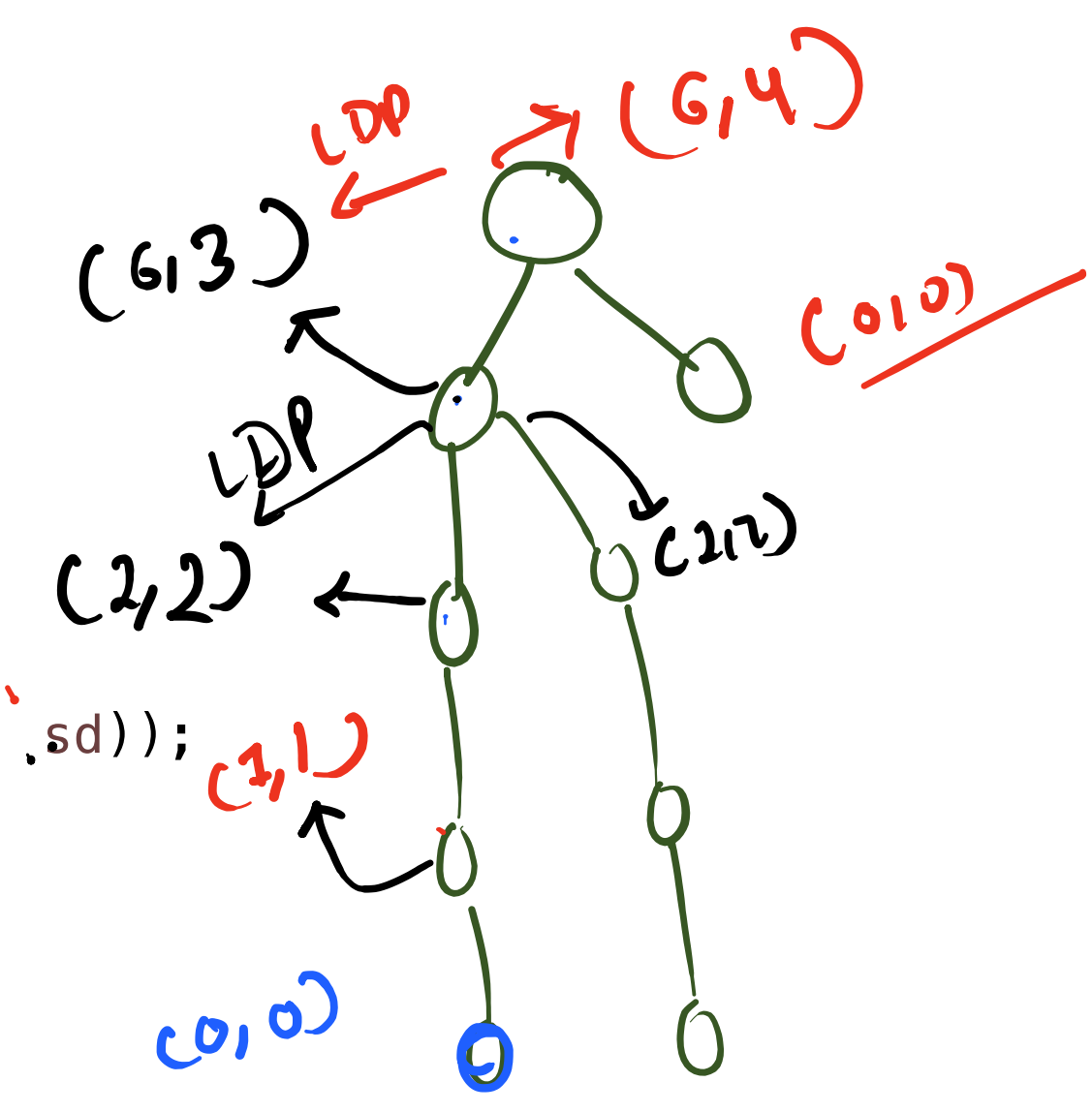
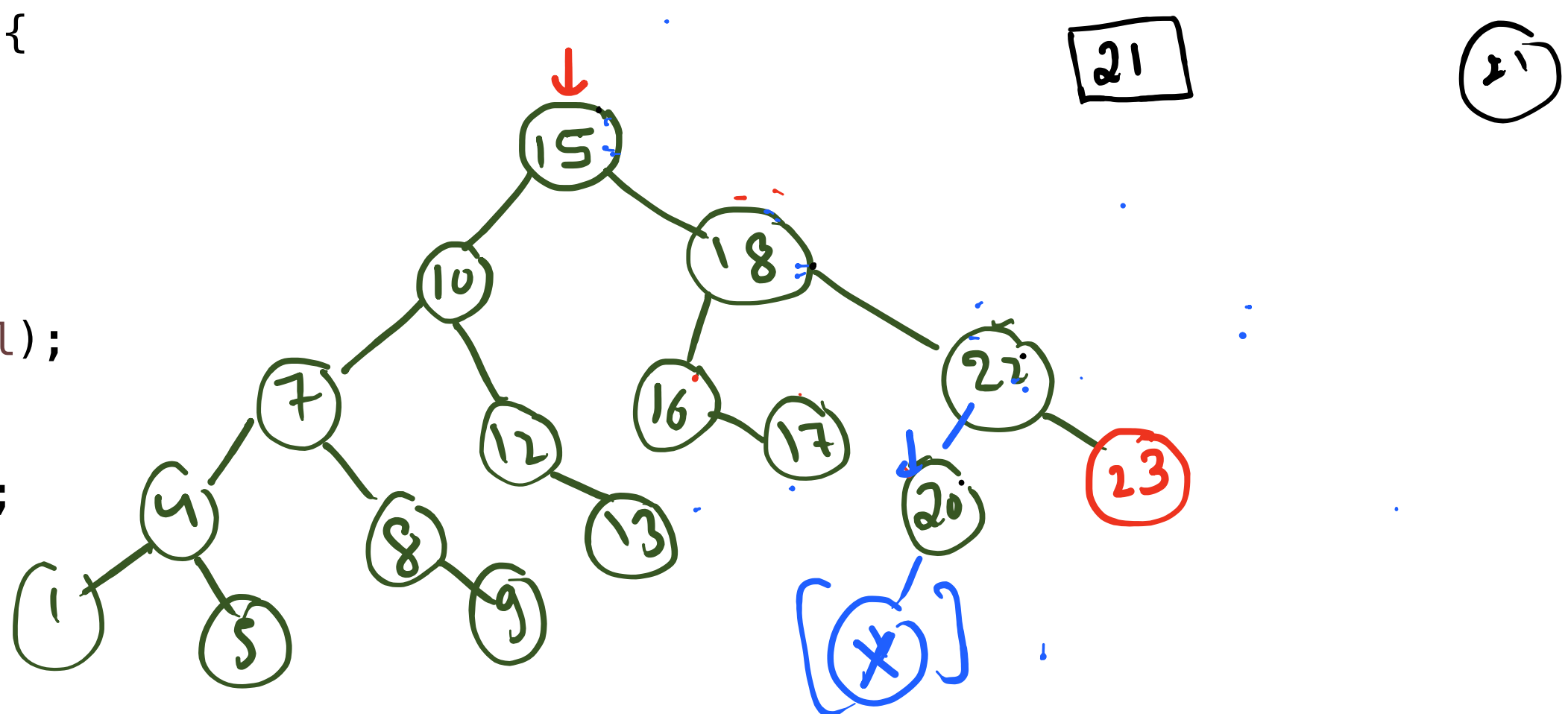


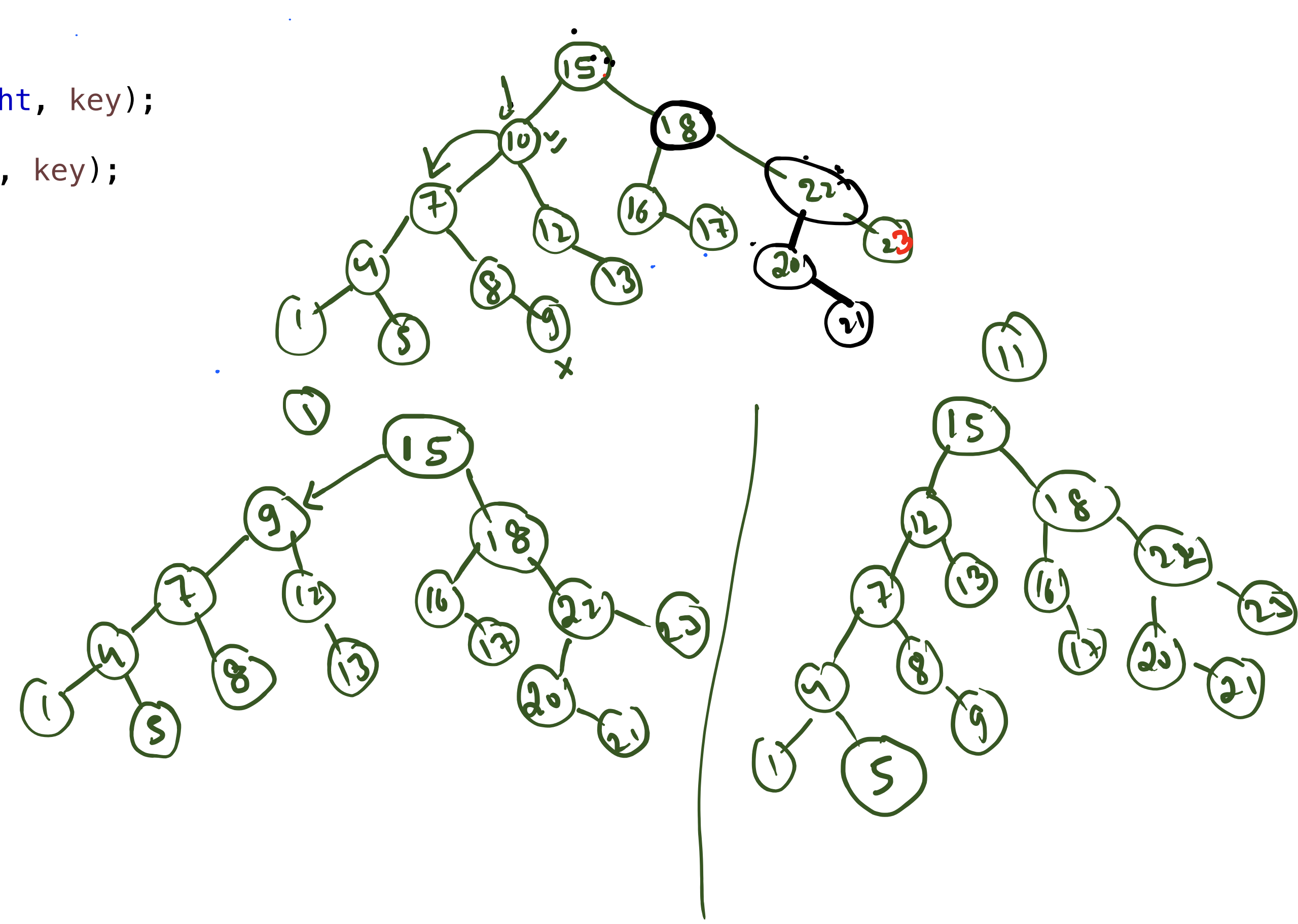
```
public DiaPair diameter(TreeNode root) {  
    if (root == null) {  
        return new DiaPair();  
    }  
    DiaPair ldp = diameter(root.left);  
    DiaPair rdp = diameter(root.right);  
    int sd = ldp.ht + rdp.ht + 2;  
    DiaPair sdp = new DiaPair();  
    sdp.dia = Math.max(ldp.dia, Math.max(rdp.dia, sd));  
    sdp.ht = Math.max(ldp.ht, rdp.ht) + 1;  
    return sdp;  
}
```



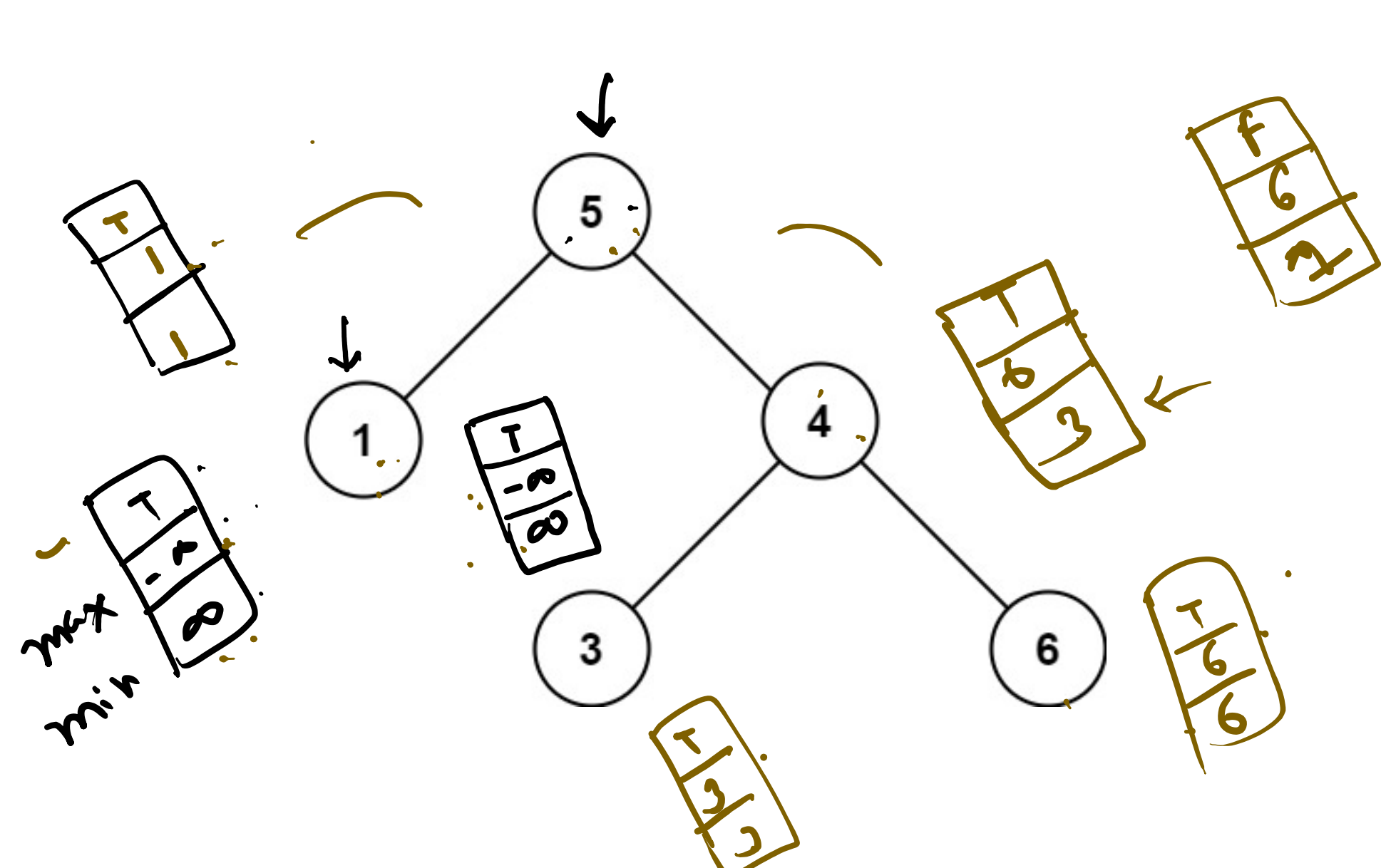
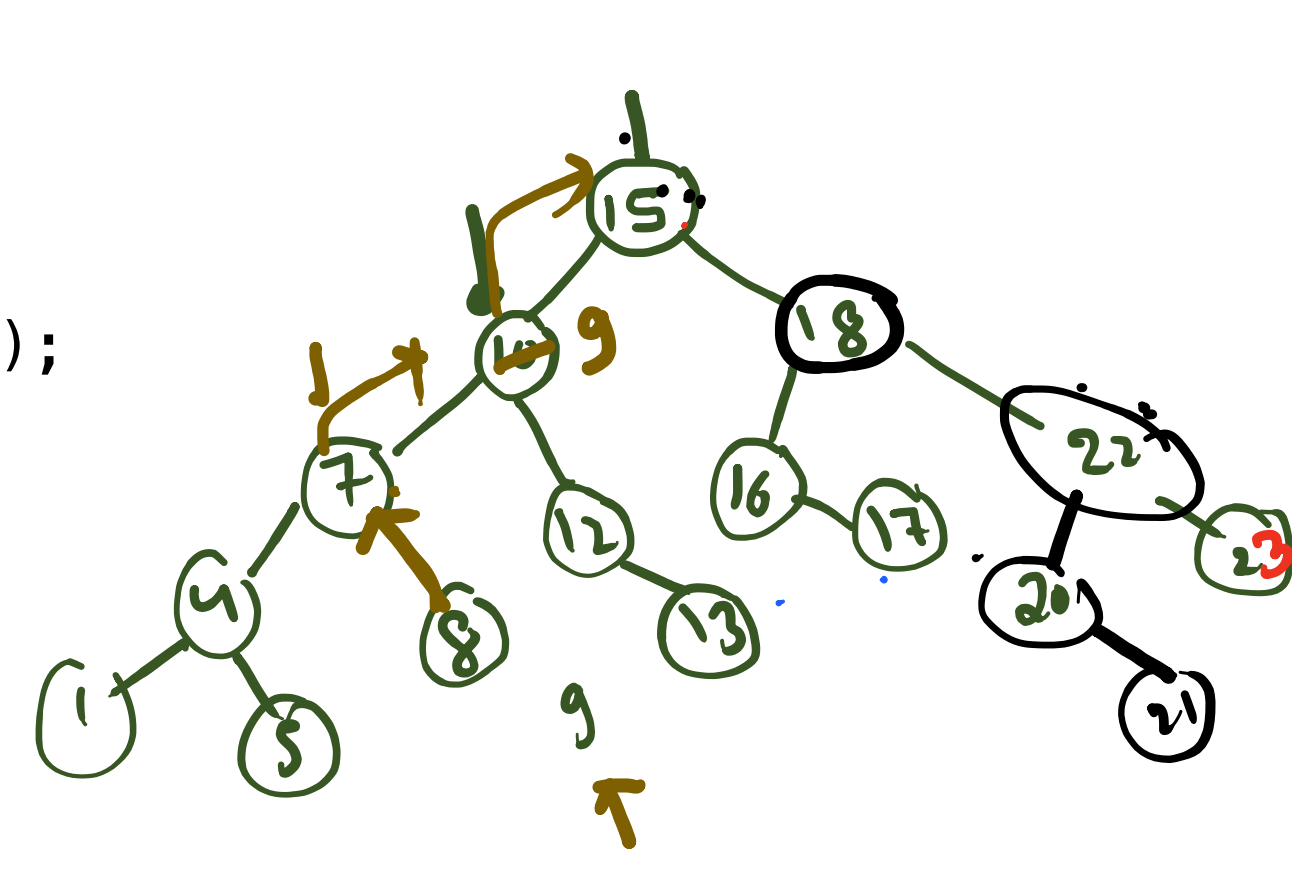
```
public TreeNode insertIntoBST(TreeNode root, int val) {  
    if (root == null) {  
        return new TreeNode(val);  
    }  
    if (root.val < val) {  
        root.right = insertIntoBST(root.right, val);  
    } else {  
        root.left = insertIntoBST(root.left, val);  
    }  
    return root;  
}
```



```
public TreeNode deleteNode(TreeNode root, int key) {  
    if (root == null) {  
        return null;  
    }  
    if (root.val < key) {  
        root.right = deleteNode(root.right, key);  
    } else if (root.val > key) {  
        root.left = deleteNode(root.left, key);  
    } else {  
        // 1 child  
        if (root.left == null) {  
            return root.right;  
        }  
        else if (root.right == null) {  
            return root.left;  
        }  
    }  
}
```

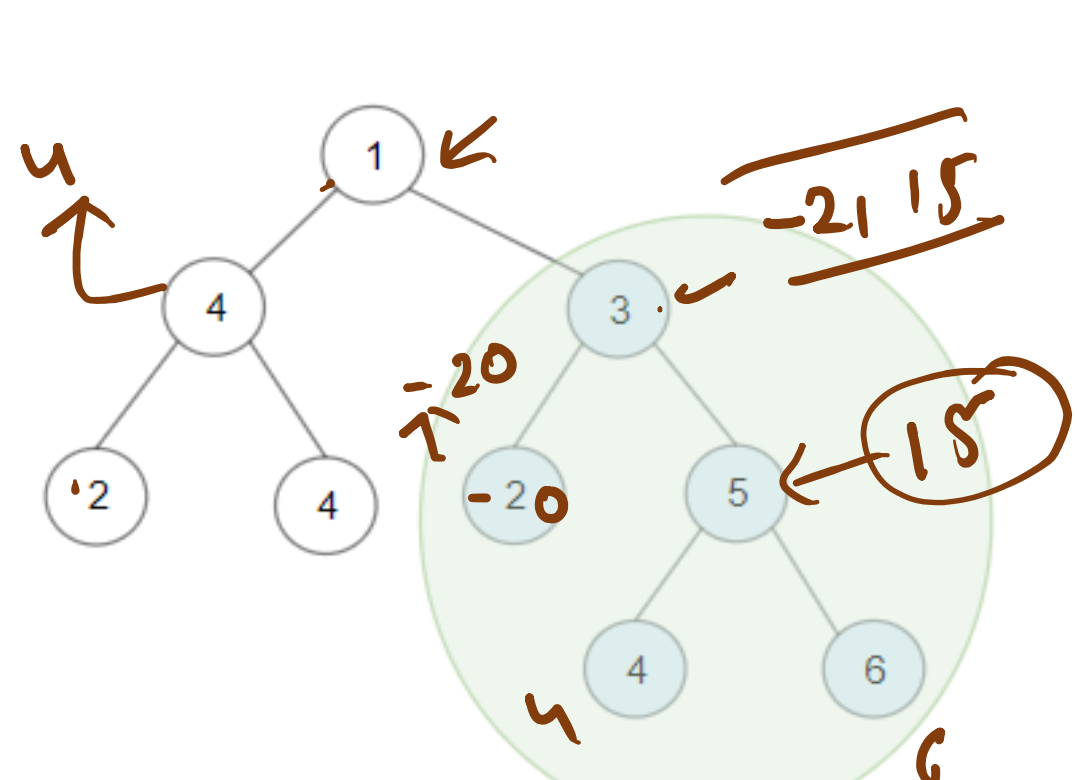
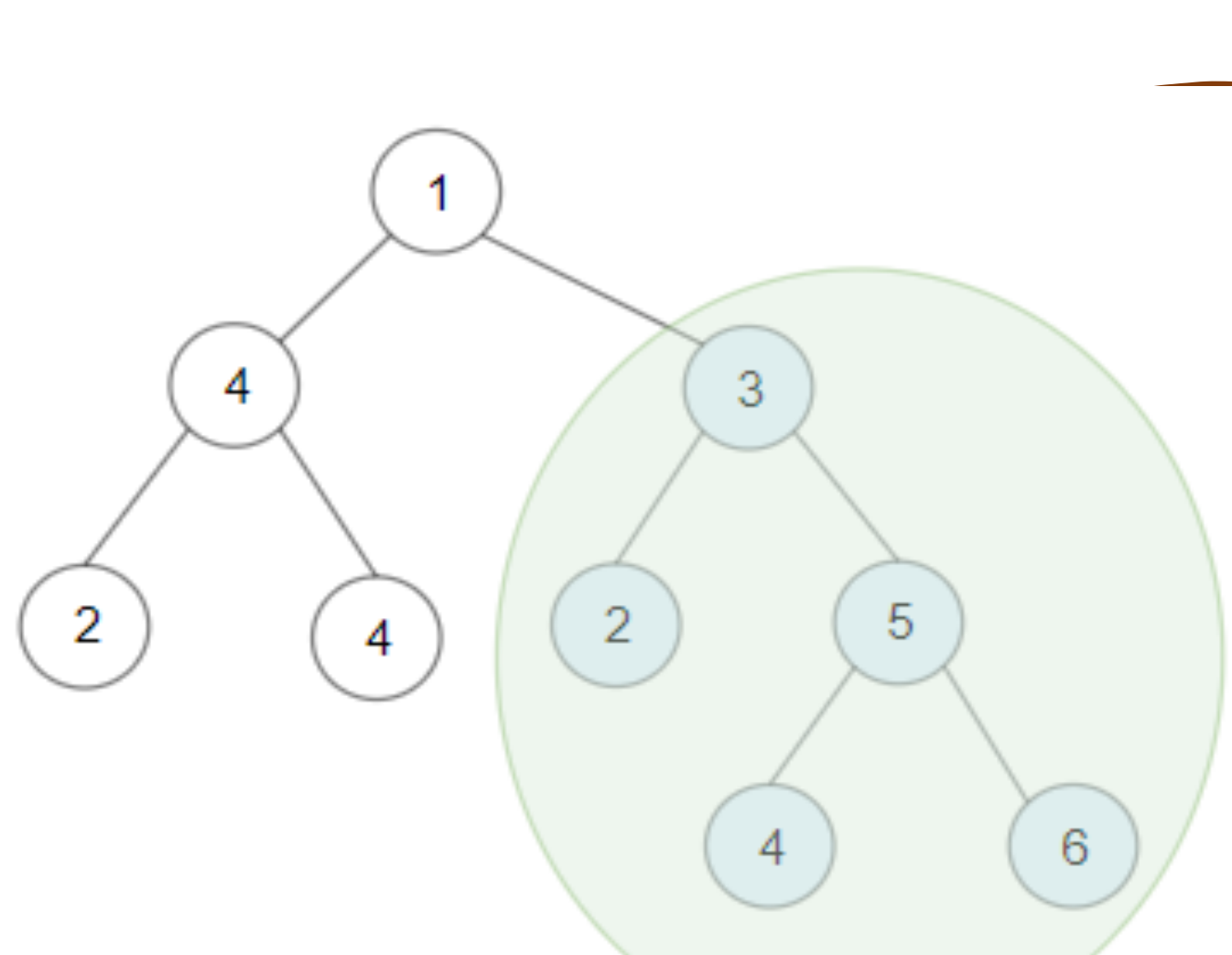


```
public TreeNode deleteNode(TreeNode root, int key) {  
    if (root == null) {  
        return null;  
    }  
    if (root.val < key) {  
        root.right = deleteNode(root.right, key);  
    } else if (root.val > key) {  
        root.left = deleteNode(root.left, key);  
    } else {  
        // 1 or 0 child  
        if (root.left == null) {  
            return root.right;  
        }  
        else if (root.right == null) {  
            return root.left;  
        }  
        else {  
            int max = max(root.left);  
            root.left = deleteNode(root.left, max);  
            root.val = max;  
        }  
    }  
    return root;  
}
```



BST pairs
is BST = true
max = Long_min val
min = Long_max val

TL TL KR (3>1)
TL TL -∞ < 1 ∞ > 1
TL TL 3 < 4 ✓ 6 > 4 ✓



```
public BstPair isBST(TreeNode root) {  
    if (root == null) {  
        return new BstPair();  
    }  
    BstPair lbp = isBST(root.left);  
    BstPair rbp = isBST(root.right);  
    BstPair sbp = new BstPair();  
    sbp.max = Math.max(lbp.max, Math.max(rbp.max, root.val));  
    sbp.min = Math.min(lbp.min, Math.min(rbp.min, root.val));  
    sbp.sum = lbp.sum + rbp.sum + root.val;  
    if (lbp.isbst && rbp.isbst && lbp.max < root.val && rbp.min > root.val) {  
        sbp.isbst = true;  
        sbp.ans = Math.max(sbp.sum, Math.max(lbp.ans, rbp.ans));  
    } else {  
        sbp.isbst = false;  
        sbp.ans = Math.max(lbp.ans, rbp.ans);  
    }  
    return sbp;  
}
```